

1. Introduction

The **Maze Solver Software** is a Java-based interactive desktop application developed as a project for the Software Development I course. The main purpose of this software is to provide an interactive environment where a user can navigate a player through a maze from a starting position to a goal position.

The application uses Java Swing to create the graphical user interface and follows Object-Oriented Programming concepts to organize the different components of the system. The software provides a graphical maze board, player movement, keyboard controls, movement validation, move counter, timer, restart functionality, automatic maze solving and goal detection.

The project was developed incrementally. Different features were implemented, tested and improved step by step throughout the development period.



Figure 2: Maze Solver Game Interface

2. Project Objectives

The main objectives of the Maze Solver Software are:

- To develop an interactive maze-solving application using Java.
 - To create a simple and user-friendly graphical user interface.
 - To display maze walls, paths, start position and goal position.
 - To allow the player to move using keyboard arrow keys.
 - To validate player movement inside the maze.
 - To prevent movement through walls.
 - To count the number of player movements.
 - To display the elapsed playing time.
 - To provide a restart facility.
 - To detect when the player reaches the goal.
 - To display a success message after completing the maze.
 - To implement an automatic maze-solving feature.
 - To apply Object-Oriented Programming concepts.
 - To use GitHub for source-code management and project development.
-

3. Project Scope

The project focuses on developing a desktop-based maze-solving application using Java and Java Swing.

The user can start the application from the main menu and enter the maze game. Inside the game, the player can move through the available paths using the keyboard. The system validates movement so that the player cannot pass through walls.

The software also provides additional features such as a timer, move counter, restart option, automatic solver and goal detection. The project does not require a database and is designed to run as a standalone Java application.

4. Development Methodology

The project was developed using the **Incremental Development Model**. In this model, the complete system is divided into smaller increments, and each increment adds a specific functionality to the software.

The project was planned and developed over ten weeks.

Week Development Work

Week 1 Project Planning and Requirement Analysis

Week 2 Software Design

Week 3 User Interface Development

Week 4 Maze Board Development

Week 5 Player Movement Implementation

Week 6 Maze Validation System

Week 7 Software Features Enhancement

Week Development Work

Week 8 Auto Solver Development

Week 9 Software Finalization

Week 10 Testing and Final Release

5. Software and Hardware Requirements

6.1 Software Requirements

The following software tools are required to develop and run the project:

- Java Development Kit (JDK)
- NetBeans IDE
- Java Swing
- GitHub
- Windows

6.2 Hardware Requirements

The application does not require high-end hardware. A normal computer or laptop capable of running Java and NetBeans is sufficient.

Basic requirements include:

- Computer or Laptop
 - Keyboard
 - Sufficient RAM
 - Sufficient storage space
-

7. System Features

The major features of the Maze Solver Software are described below.

7.1 Main Menu

The Main Menu is the starting interface of the application. It provides the basic options required to use the software.

The main options are:

- Start
- Help
- Exit

The **Start** button opens the maze game, **Help** provides basic instructions and **Exit** closes the application.



Figure 4: Main Menu of the Application

7.2 Maze Board

The Maze Board is the main gaming area of the application. It displays the maze using a graphical grid containing walls and paths.

The board also identifies the starting position and goal position so that the user can understand the objective of the game.



Figure 5: Maze Board with Walls, Paths, Start and Goal

7.3 Player Movement

The player can navigate through the maze using the keyboard arrow keys.

The controls are:

- **Up Arrow** — Move Up
- **Down Arrow** — Move Down
- **Left Arrow** — Move Left
- **Right Arrow** — Move Right

The player's row and column position are updated according to the selected direction.



Figure 6: Player Movement Inside the Maze

7.4 Movement Validation

The validation system checks whether the requested movement is allowed. If the target position is a wall or an invalid position, the player cannot move into that position.

This prevents the player from passing through the maze walls and ensures that the game rules are maintained.

7.5 Move Counter

The Move Counter keeps track of the number of valid movements made by the player during the game.

The counter provides information about how many movements the user has taken to solve the maze.



Figure 7: Move Counter

7.6 Timer

The Timer records the elapsed time while the user is playing the maze. It helps the user monitor the time required to complete the maze.

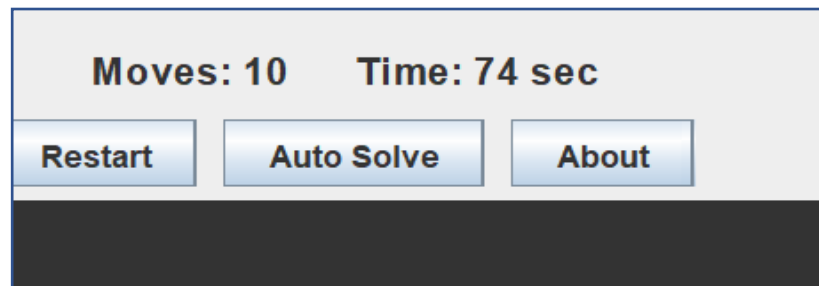


Figure 9: Game Timer

7.7 Restart Function

The Restart feature allows the user to start the maze again without closing the application. When the game is restarted, the player returns to the initial position and the game state is reset.



Figure 10: Restart Function

7.8 Automatic Solver

The Auto Solver provides an automatic way to solve the maze. The solver searches through the available paths from the starting position and attempts to find a valid path to the goal. The project uses maze-solving and pathfinding concepts such as DFS/BFS as part of the automatic solving development.



Figure 11: Automatic Maze Solver

7.9 Goal Detection

The software checks whether the player has reached the defined goal position.

When the player reaches the goal, the system identifies that the maze has been completed and displays a success message.



Figure 12: Goal Detection and Success Message

8. System Architecture

The system is divided into several classes and functional modules.

Main Components

MazeGame.java

This class starts the application.

MainMenu.java

This class provides the main menu and navigation options.

MazeBoard.java

This class manages the maze board, graphical interface and player interaction.

Player.java

This class stores and updates the player's row and column position.

Validation Module

Checks whether the player's requested movement is valid.

Timer Module

Keeps track of the elapsed game time.

Move Counter Module

Counts the player's valid movements.

Restart Module

Resets the game state.

Auto Solver Module

Searches for a valid path from the starting position to the goal.

Goal Detection Module

Checks whether the player has reached the goal.

9. Project Structure

The Java source files are organized inside the mazegame package.

The basic project structure is:

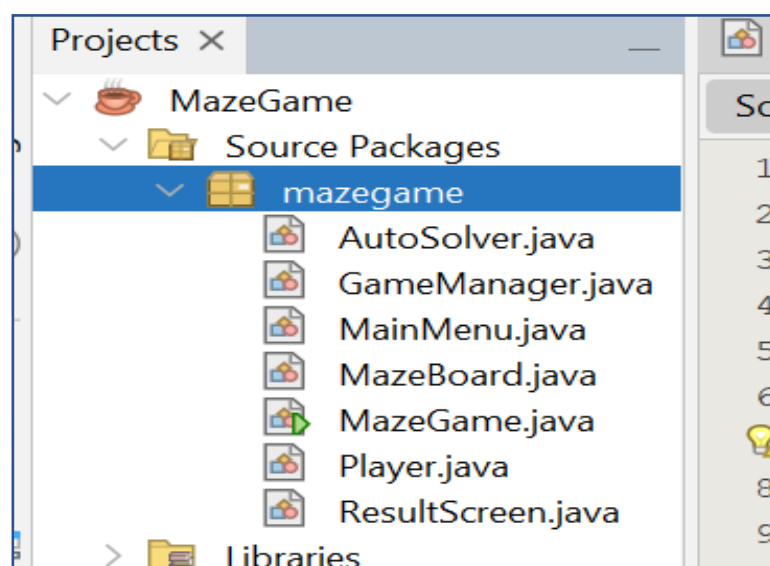


Figure 14: Project Structure in NetBeans

10. Object-Oriented Programming Concepts Used

The project applies several important Object-Oriented Programming concepts.

10.1 Class and Object

The application is divided into classes such as MazeGame, MainMenu, MazeBoard and Player. Objects are created from these classes to perform different operations.

10.2 Encapsulation

The Player class stores player position using private variables. The position can be accessed through methods such as `getRow()` and `getCol()`.

10.3 Inheritance

The GUI classes inherit from JFrame.

For example:

```
public class MainMenu extends JFrame
```

and

```
public class MazeBoard extends JFrame
```

10.4 Modularity

Different responsibilities are divided among different classes and modules, making the system easier to develop and maintain.

11. Data Structure Used

The maze is represented using a **two-dimensional integer array**.

The basic representation is:

1 = Wall

0 = Path

For example:

```
1 1 1 1 1
```

```
1 0 0 1
```

```
1 0 1 0 1
```

```
1 0 0 1
```

```
1 1 1 1 1
```

The player's position is maintained using row and column coordinates.

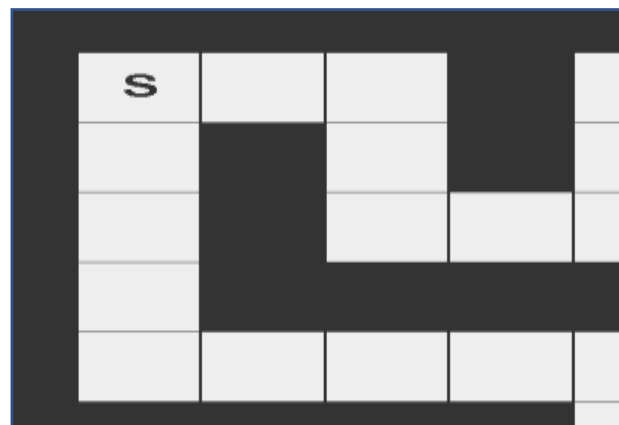


Figure 15: Two-Dimensional Maze Representation

12. Game Rules

The basic rules of the maze are:

1. The player starts from the defined starting position.
2. The player can move through valid path cells.
3. The player cannot move through wall cells.
4. Arrow keys are used for movement.
5. Invalid movements are prevented.
6. The objective is to reach the goal.
7. The game is completed when the player reaches the goal.

13. Game Flow

The overall game flow of the software is:

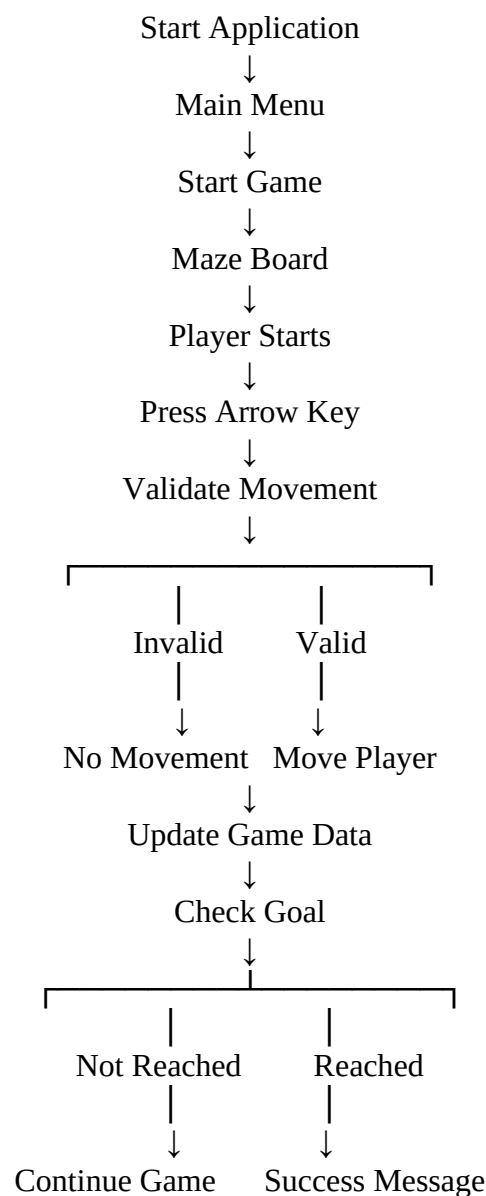


Figure 16: Game Flow of Maze Solver Software

14. Testing

Testing was performed to verify the functionality of the software.

Test Case	Expected Result
Start Application	Main Menu opens
Start Game	Maze Board opens
Valid Movement	Player moves
Wall Movement	Movement is prevented
Move Counter	Counter increases
Timer	Time increases
Restart	Initial state restored
Auto Solve	Maze is solved
Reach Goal	Success message appears

Testing helped identify and fix errors during development.

15. Error Handling and Validation

The software validates player actions before updating the player's position.

The system checks:

- Whether the target cell is a wall.
- Whether the target position is outside the maze.
- Whether the requested movement is valid.
- Whether the player has reached the goal.

If the requested movement is invalid, the player remains at the current valid position.

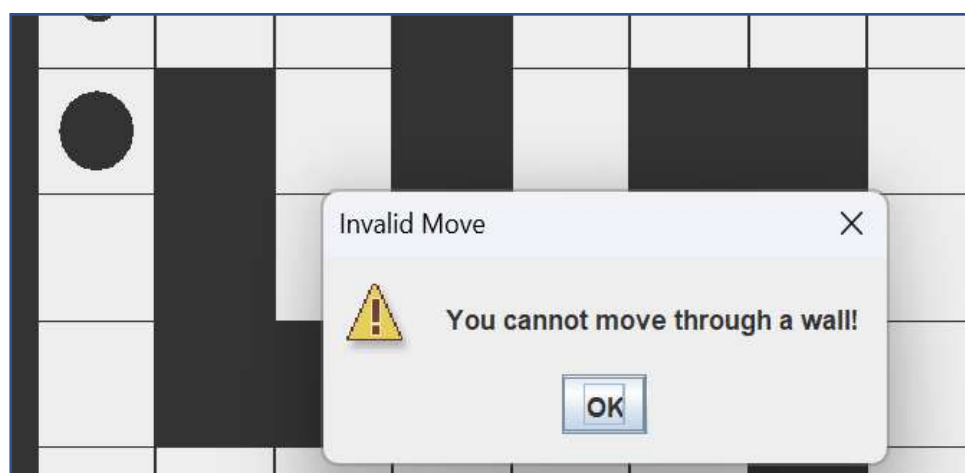


Figure 17: Movement Validation

16. GitHub Version Control

GitHub was used to store and manage the source code and project documentation.

During development, different modules and project files were uploaded to the GitHub repository according to the development schedule.

The final repository contains the source code and supporting project materials such as documentation, diagrams, screenshots and user manual.

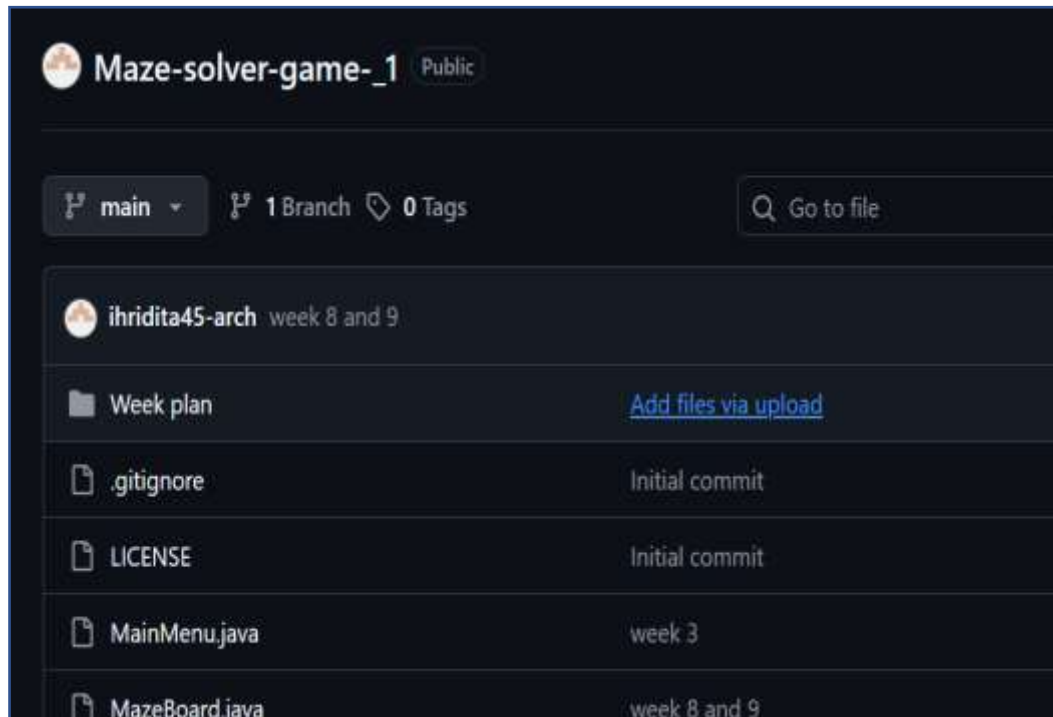


Figure 19: Maze Solver Software GitHub Repository

17. Downloading the Project from GitHub

Another user can download and use the Maze Solver Software from the GitHub repository.

The download process is:

Step 1: Open GitHub Repository

Open the Maze Solver Software repository on GitHub.

Step 2: Click Code

Click the **Code** button on the repository page.

Step 3: Download ZIP

Select **Download ZIP** and save the project to the computer.

Step 4: Extract the ZIP File

Extract the downloaded ZIP file.

\

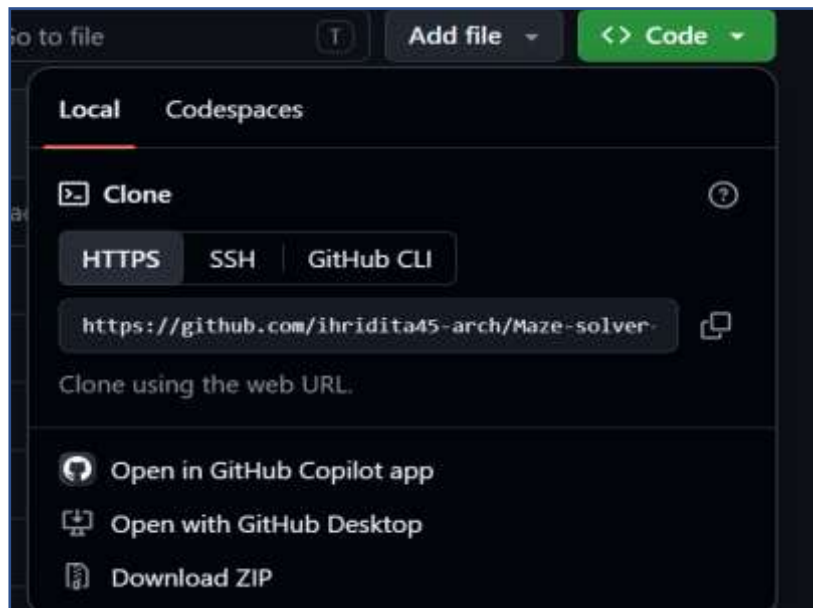


Figure 20: Downloading Maze Solver Software from GitHub

18. Opening the Project in NetBeans

After downloading and extracting the project, the user can open it in NetBeans.

The steps are:

1. Open NetBeans IDE.
2. Select **File** → **Open Project**.
3. Select the extracted Maze Solver project folder.
4. Click **Open Project**.
5. Wait for the project to load.
6. Check the mazegame package and source files.

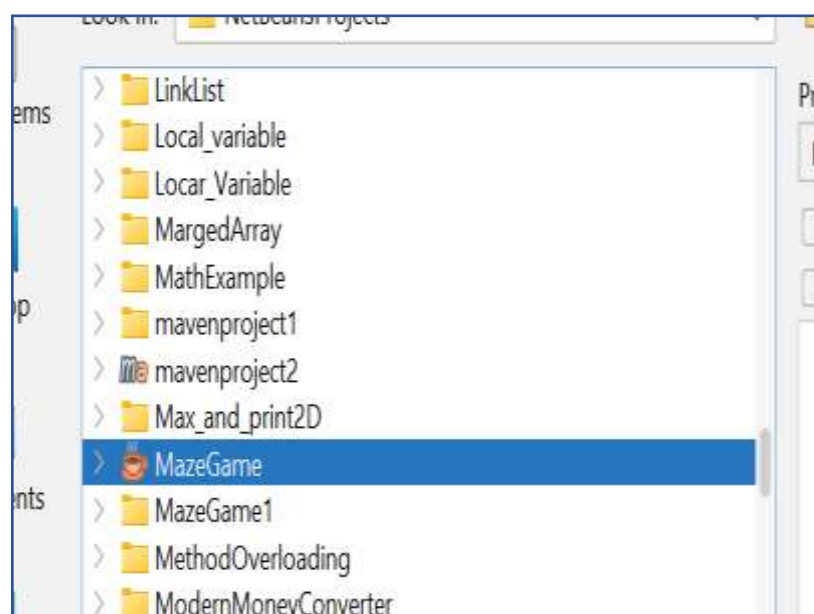


Figure 21: Opening the Project in NetBeans

19. Running the Project

After opening the project in NetBeans:

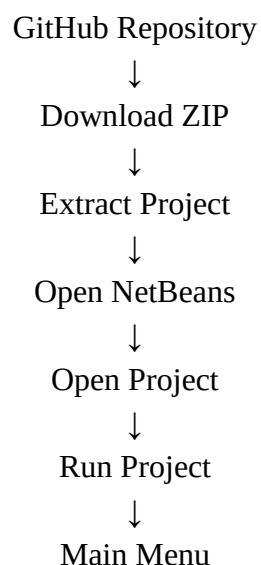
1. Run the project.
2. The Main Menu will appear.
3. Click **Start**.
4. The Maze Board will open.
5. Use the arrow keys to control the player.
6. Navigate through the maze.
7. Reach the Goal to complete the game.



Figure 22: Running the Maze Solver Software

20. How Another User Can Use the Software

The complete process for another user is:





Therefore, another user does not need to create the project from the beginning. The user can download the source code from GitHub, open it in NetBeans and run the application.

21. Challenges Faced

During the development of the Maze Solver Software, several challenges were faced.

The major challenges included:

- Designing the maze structure.
- Representing the maze using a two-dimensional array.
- Creating the graphical maze board.
- Implementing keyboard controls.
- Connecting GUI with game logic.
- Implementing wall collision detection.
- Updating player position correctly.
- Managing timer and move counter.
- Implementing restart functionality.
- Developing the automatic solver.
- Detecting the goal.
- Testing and debugging the complete application.

These challenges provided practical experience in Java programming and software development.

22. Result

The Maze Solver Software was successfully developed as an interactive Java Swing application.

The final application provides a graphical maze where the user can control a player using keyboard arrow keys. The system validates movement, prevents invalid movement, counts moves, records playing time and provides restart and automatic solving facilities.

The project also demonstrates the practical use of Java, Object-Oriented Programming, Java Swing, event handling, validation, two-dimensional arrays, pathfinding concepts and GitHub version control.



Figure 24: Final Maze Solver Software

23. Future Improvements

The software can be improved in future versions by adding:

- Multiple maze levels.
 - Different maze sizes.
 - Difficulty levels.
 - Random maze generation.
 - Sound effects.
 - Score system.
 - Player statistics.
 - Better animations.
 - Improved graphical design.
 - Dark mode.
 - Additional maze-solving algorithms.
 - Save and load functionality.
-

24. Conclusion

The Maze Solver Software project provided practical experience in the complete software development process. The project started with requirement analysis and design and was gradually developed through GUI development, maze board creation, player movement, validation, feature enhancement, automatic solving, goal detection, testing and final documentation.

The project demonstrates the practical application of Java, Object-Oriented Programming, Java Swing, event handling, validation, data structures, pathfinding concepts and GitHub version control.

The final application provides an interactive environment where users can manually solve the maze and use the automatic solver to find a solution.

25. Project Demonstration

During the final project demonstration, the following features will be shown:

1. Launching the application.
2. Main Menu.
3. Start Game.
4. Maze Board.
5. Player movement using arrow keys.
6. Wall collision and movement validation.
7. Move Counter.
8. Timer.
9. Restart functionality.
10. Auto Solver.
11. Goal detection.
12. Success message.
13. GitHub repository.
14. Downloading the project from GitHub.
15. Opening the project in NetBeans.
16. Running the project.

26. Final Release

The final version of the Maze Solver Software is prepared as **Version 1.0**.

The final project repository contains:

- Final Source Code
- Project Documentation
- Software Design Document
- Class Diagram
- Workflow Diagram
- User Manual
- Screenshots
- README file
- Final Project Files

The project can be downloaded from GitHub and opened in NetBeans for execution and further development.